

FETCH_DASHBOARD-DATA.PHP

```
<?php
session_start();
header('Content-Type: application/json');
include '../includes/db_connect.php';

if (!isset($_SESSION['user_id'])) {
    echo json_encode(["error" => "Not logged in"]);
    exit;
}

$userRole = $_SESSION['role'] ?? 'guest';
$orgId = $_SESSION['org_id'] ?? null;

try {
    if ($userRole === 'osas' || $userRole === 'admin') {
        $query = "SELECT a.*, o.name AS org_name,
            (SELECT COUNT(*) FROM documents d WHERE d.activity_id = a.activity_id) AS
doc_count
            FROM activities a
            JOIN organizations o ON a.org_id = o.org_id
            ORDER BY a.created_at DESC";
        $result = pg_query($conn, $query);
    } else if ($orgId) {
        $query = "SELECT a.*, o.name AS org_name,
            (SELECT COUNT(*) FROM documents d WHERE d.activity_id = a.activity_id) AS
doc_count
            FROM activities a
            JOIN organizations o ON a.org_id = o.org_id
            WHERE a.org_id = $1
            ORDER BY a.created_at DESC";
        $result = pg_query_params($conn, $query, [$orgId]);
    } else {
        echo json_encode(["error" => "No organization ID found"]);
        exit;
    }
}

$activities = pg_fetch_all($result) ?: [];

// Normalize doc_count to int if present
foreach ($activities as &$act) {
    if (isset($act['doc_count'])) {
        $act['doc_count'] = (int)$act['doc_count'];
    } else {
```

```

        $act['doc_count'] = 0;
    }
}
unset($act);

// Get documents list (recent) — either all (admin) or for the org
if ($UserRole === 'osas' || $UserRole === 'admin') {
    $docQuery = "SELECT d.*, o.name AS org_name, a.name AS activity_name
        FROM documents d
        LEFT JOIN organizations o ON d.org_id = o.org_id
        LEFT JOIN activities a ON d.activity_id = a.activity_id
        ORDER BY d.uploaded_at DESC
        LIMIT 50";
    $docResult = pg_query($conn, $docQuery);
} else {
    $docQuery = "SELECT d.*, o.name AS org_name, a.name AS activity_name
        FROM documents d
        LEFT JOIN organizations o ON d.org_id = o.org_id
        LEFT JOIN activities a ON d.activity_id = a.activity_id
        WHERE d.org_id = $1
        ORDER BY d.uploaded_at DESC
        LIMIT 50";
    $docResult = pg_query_params($conn, $docQuery, [$orgId]);
}

$documents = pg_fetch_all($docResult) ?: [];

// Count documents
if ($UserRole === 'osas' || $UserRole === 'admin') {
    $docCountRes = pg_query($conn, "SELECT COUNT(*) FROM documents");
} else {
    $docCountRes = pg_query_params($conn, "SELECT COUNT(*) FROM documents
WHERE org_id = $1", [$orgId]);
}
$totalDocs = (int)pg_fetch_result($docCountRes, 0, 0);

echo json_encode([
    "activities" => $activities,
    "documents" => $documents,
    "totalActivities" => count($activities),
    "totalDocuments" => $totalDocs
]);

} catch (Exception $e) {

```

```
    echo json_encode(["error" => $e->getMessage()]);  
}  
?>
```

1. fetch_dashboard_data.php — Dashboard API

Purpose:

- Returns **JSON** data about:
 - Activities
 - Documents
 - Total counts
- Serves the `dashboard.js` frontend.

Key parts & what you should know:

Part	Explanation
<code>session_start()</code>	Start session to read user info.
<code>header('Content-Type: application/json')</code>	Tells browser the output is JSON.
<code>\$_SESSION['role'] / \$_SESSION['org_id']</code>	User role and org id from login.
<code>\$query</code>	SQL query to fetch data.
<code>pg_query_params(\$conn, \$query, [\$orgId])</code>	Run SQL with placeholders (<code>%s</code>) safely. Prevents SQL injection.
<code>\$result</code>	The result set from PostgreSQL query (not yet usable as array).
<code>pg_fetch_all(\$result)</code>	Converts <code>\$result</code> to an array of associative arrays (each row is an array).
<code>\$activities / \$documents</code>	Arrays of activities or documents fetched from DB.
<code>(int)\$act['doc_count']</code>	Converts PostgreSQL counts (strings) to integers.
<code>echo json_encode(...)</code>	Sends JSON to JavaScript.

What `$activities` means:

- `$activities` is an **array of associative arrays**, each representing one activity from the DB.
- Example:

What `$activities` means:

- `$activities` is an array of associative arrays, each representing one activity from the DB.
- Example:

php

 Copy code

```
$activities = [  
    ["activity_id"=>"1", "name"=>"Orientation", "doc_count"=>5, "org_name"=>"CS Club"],  
    ["activity_id"=>"2", "name"=>"Seminar", "doc_count"=>2, "org_name"=>"CS Club"]  
];
```

Other notes:

- Admin/OSAS sees all orgs; normal org users see only their own org's activities.
- `pg_query_params()` → use when variables are in SQL.
- `pg_fetch_all()` → fetch all rows at once.
- `pg_fetch_result()` → fetch a single value (used for total documents).

LOGIN.PHP

```
<?php  
session_start();  
include '../includes/db_connect.php';  
  
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    $email = trim($_POST['email']);  
    $password = trim($_POST['password']);  
  
    // Parameterized query  
    $query = "SELECT * FROM users WHERE email = $1 LIMIT 1";  
    $result = pg_query_params($conn, $query, array($email));  
  
    if ($result && pg_num_rows($result) === 1) {  
        $user = pg_fetch_assoc($result);  
  
        // Plain password compare (existing behavior)  
        if ($password === $user['password']) {  
            $_SESSION['user_id'] = $user['user_id'];  
            $_SESSION['org_id'] = $user['org_id'];  
            $_SESSION['name'] = $user['name'];  
            $_SESSION['role'] = $user['role'];  
  
            // If user has an org_id, fetch the organization name for display in header
```

```

if (!empty($user['org_id'])) {
    $orgQuery = "SELECT name FROM organizations WHERE org_id = $1 LIMIT 1";
    $orgRes = pg_query_params($conn, $orgQuery, array($user['org_id']));
    if ($orgRes && pg_num_rows($orgRes) === 1) {
        $_SESSION['org_name'] = pg_fetch_result($orgRes, 0, 0);
    }
}

header("Location: ../html/dashboard.php");
exit;
} else {
    header("Location: ../html/login.html?error=Incorrect password");
    exit;
}
} else {
    header("Location: ../html/login.html?error=Email not found");
    exit;
}
} else {
    header("Location: ../html/login.html");
    exit;
}
?>

```

2. login.php — User login

Purpose:

- Receives login form POST data
- Checks email and password
- Stores session variables for later pages
- Redirects to dashboard

Key points:

Part	Explanation
<code>\$_POST['email'], \$_POST['password']</code>	User input from HTML form
<code>pg_query_params()</code>	Prevent SQL injection by using <code>%s</code> placeholder
<code>\$user = pg_fetch_assoc(\$result)</code>	Fetch single row as associative array (key = column name)
<code>\$_SESSION['user_id'], \$_SESSION['role'], etc.</code>	Session variables saved for later use
<code>header("Location: dashboard.php")</code>	Redirect user after login

Why `$user` ?

- `$user` holds one database row, e.g., `["user_id"=>1,"name"=>"John","role"=>"admin"]`.

LOGOUT.PHP

```
<?php
session_start();
session_destroy();
header("Location: ../html/login.html");
exit;
?>
```

3. logout.php — Log out

- Simple:

php

 Copy code

```
session_start();
session_destroy(); // removes all session variables
header("Location: login.html"); // back to login page
```

DASHBOARD.JS

```
document.addEventListener("DOMContentLoaded", () => {
  const totalActivities = document.getElementById("totalActivities");
  const totalDocuments = document.getElementById("totalDocuments");
  const documentsElem = document.getElementById("documents");
  const recentActivities = document.getElementById("activities");
  const modal = document.getElementById("modal");
  const modalTitle = document.getElementById("modalTitle");
  const modalContent = document.getElementById("modalContent");
  const closeModal = document.getElementById("closeModal");
```

// Close modal

```
closeModal.addEventListener("click", () => {
  modal.style.display = "none";
});
```

// Function to fetch dashboard data

```
function loadDashboardData() {
  fetch("../php/fetch_dashboard_data.php")
    .then(res => res.json())
    .then(data => {
      if (data.error) {
        console.error("Dashboard API error:", data.error);
        return;
      }
    })
```

```
totalActivities.textContent = data.totalActivities;
totalDocuments.textContent = data.totalDocuments;
```

```
documentsElem.innerHTML = "";
recentActivities.innerHTML = "";
```

```

// Render activities
(data.activities || []).forEach(act => {
  const div = document.createElement("div");
  div.className = "activity-item";
  div.style.padding = "10px";
  div.style.borderBottom = "1px solid #ccc";
  div.style.cursor = "pointer";
  div.innerHTML = `
    <div class="activity-two-col">
      <div style="overflow:hidden; text-overflow:ellipsis;"><strong>Name:</strong>
${act.name}</div>
      <div><strong>Docs:</strong> ${act.doc_count || 0}</div>
    </div>
    <div class="activity-two-col">
      <div><strong>Academic Year:</strong> ${act.academic_year || "N/A"}</div>
      <div><strong>SDG:</strong> ${act.sdg_relation || "N/A"}</div>
    </div>
  `;

  div.addEventListener("click", () => {
    modal.style.display = "flex";
    modalTitle.textContent = act.name;
    modalContent.innerHTML = `
      <p><strong>Organization:</strong> ${act.org_name || "N/A"}</p>
      <p><strong>SDG:</strong> ${act.sdg_relation || "N/A"}</p>
      <p><strong>Academic Year:</strong> ${act.academic_year || "N/A"}</p>
      <p><strong>Semester:</strong> ${act.semester || "N/A"}</p>
      <p><strong>Date:</strong> ${act.date_started || "N/A"} to ${act.date_ended ||
"N/A"}</p>
      <p><strong>Description:</strong><br>${act.description || "No description
provided."}</p>
    `;
  });

  recentActivities.appendChild(div);
});

// Render documents
(data.documents || []).forEach(doc => {
  const d = document.createElement("div");
  d.className = "doc-item";
  d.innerHTML = `
    <div class="activity-two-col">

```

```

        <div style="overflow:hidden;
text-overflow:ellipsis;"><strong>${doc.document_name}</strong></div>
        <div>${doc.document_type || ""}</div>
    </div>
    <div class="activity-two-col">
        <div><small>${doc.activity_name ? "Activity: " + doc.activity_name : ""}</small></div>
        <div><small>${doc.org_name ? doc.org_name : ""}</small></div>
    </div>
    `;
    documentsElem.appendChild(d);
});

})
.catch(err => console.error("Fetch error:", err));
}

// Initial load
loadDashboardData();

// Auto-refresh if a new activity was added
if (localStorage.getItem("activityAdded") === "true") {
    loadDashboardData(); // Refresh data immediately
    localStorage.removeItem("activityAdded");
}

// Real-time refresh if another tab adds new activity
window.addEventListener("storage", (event) => {
    if (event.key === "activityAdded" && event.newValue === "true") {
        loadDashboardData();
        localStorage.removeItem("activityAdded");
    }
});
});

```

5. dashboard.js — Frontend JS

Purpose:

- Fetches dashboard data from PHP API
- Displays activities and documents
- Handles modal popups
- Handles auto-refresh when new activities are added

Key points:

Part	Explanation
<code>fetch("../php/fetch_dashboard_data.php")</code>	Sends GET request to PHP API
<code>.then(res => res.json())</code>	Converts response to JavaScript object
<code>data.activities</code> , <code>data.documents</code>	Arrays of activities & documents received from PHP
<code>createElement("div")</code>	Dynamically creates HTML elements
<code>addEventListener("click")</code>	Open modal to show activity details
<code>localStorage</code> & <code>storage</code> event	Detect changes from other tabs for auto-refresh

HEADER.PHP

```
<?php
// includes/header.php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// Default labels
$roleLabel = 'Guest';
$orgLabel = 'Student Org';

if (!empty($_SESSION['role'])) {
    $roleLabel = htmlspecialchars($_SESSION['role']);
    // nice formatting: capitalize first letter
    $roleLabel = ucfirst($roleLabel);
}

if (!empty($_SESSION['org_name'])) {
```

```
$orgLabel = htmlspecialchars($_SESSION['org_name']);
}
?>
```

```
<div class="header-nav">
  
  <p><?php echo $roleLabel; ?></p>
  <p>|</p>
  <p><?php echo $orgLabel; ?></p>
</div>
```

4. header.php — Page header

Purpose:

- Shows user role and organization at top of page
- Uses session info

Key points:

Part	Explanation	Copy
<code>\$_SESSION['role']</code>	Current logged-in role	
<code>\$_SESSION['org_name']</code>	Current org	
<code>htmlspecialchars()</code>	Escape HTML to prevent XSS attacks	
<code>ucfirst()</code>	Capitalize first letter of role	
<code>\$roleLabel</code> , <code>\$orgLabel</code>	Variables used to output in HTML	

Example output:

```
html Copy code

<div class="header-nav">
  
  <p>Admin</p>
  <p>|</p>
  <p>CS Club</p>
</div>
```